



Audit organisation et technique

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à l'audit de sécurité

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

Un **audit de sécurité** est une évaluation systématique et indépendante du niveau de sécurité d'un système d'information, visant à vérifier sa conformité à un référentiel (normatif, réglementaire ou contractuel) et à identifier les écarts entre l'état constaté et l'état attendu. Le mot « systématique » est important : un audit suit une méthodologie explicite et reproductible (checklist, référentiel, grille d'évaluation), à la différence d'un simple test ponctuel ou d'une impression subjective sur la sécurité d'un système.

L'**indépendance** de l'auditeur, qu'elle soit interne (un service distinct de celui qui exploite le système) ou externe (un prestataire tiers), est ce qui donne sa crédibilité au constat : un audit réalisé par l'équipe qui a elle-même configuré le système manque structurellement d'objectivité, même si les auditeurs sont compétents et de bonne foi. C'est pour cette raison que la plupart des référentiels de certification (ISO 27001 en tête) exigent des audits menés par des personnes n'ayant pas de responsabilité directe dans la mise en œuvre des mesures évaluées.

Il faut enfin distinguer l'audit de deux activités voisines mais différentes : le **contrôle** (vérification ponctuelle d'un point précis, sans démarche globale) et le **conseil** (accompagnement à l'amélioration, sans posture d'évaluation indépendante). Un auditeur peut, dans certains contextes, recommander des solutions, mais son rôle premier reste de constater un écart entre un état observé et un état de référence, pas de concevoir lui-même la solution technique.

2. Pourquoi auditer ?

Les organisations n'auditent pas leur sécurité par pure vertu : l'audit répond en général à un ou plusieurs déclencheurs concrets, qu'il est utile de distinguer car ils orientent le périmètre, le référentiel et le niveau d'exigence attendu.

- **Conformité réglementaire** : de nombreux secteurs sont soumis à des obligations légales de protection des données ou de sécurité des systèmes (secteur bancaire, santé, opérateurs d'importance vitale). L'audit permet de démontrer, preuves à l'appui, que ces obligations sont respectées, et constitue souvent une pièce du dossier en cas de contrôle par une autorité de régulation.
- **Exigence contractuelle** : un client, un partenaire ou un assureur peut exiger un niveau de sécurité prouvé avant de signer un contrat ou de couvrir un risque cyber — par exemple une certification ISO 27001, ou une conformité PCI-DSS pour toute entité qui traite des données de cartes bancaires. Dans ce cas, l'audit n'est pas seulement un exercice interne : son résultat conditionne une relation d'affaires.
- **Gestion des risques** : l'objectif le plus intuitif est d'identifier les vulnérabilités et les faiblesses organisationnelles avant qu'un attaquant ne les découvre et ne les exploite. L'audit joue ici un rôle préventif, complémentaire de la détection en temps réel (SOC, supervision) qui, elle, intervient une fois l'attaque déjà en cours.
- **Amélioration continue** : réalisé de façon périodique, l'audit permet de mesurer objectivement les progrès (ou les régressions) entre deux évaluations successives, et de justifier ainsi des investissements en sécurité auprès de la direction avec des indicateurs concrets plutôt qu'avec un sentiment général.

Ces motivations ne s'excluent pas : un même audit répond souvent simultanément à une exigence contractuelle et à un objectif de gestion des risques, par exemple.

3. Les grandes familles d'audit

Le terme « audit de sécurité » recouvre en réalité plusieurs types d'exercices, dont l'objet, les méthodes et les livrables diffèrent sensiblement. Il est important de bien identifier de quel type d'audit on parle avant de définir un périmètre ou de choisir une méthodologie.

Type d'audit	Objet	Exemple de livrable
Audit organisationnel	politiques, procédures, gouvernance	rapport de conformité ISO 27001
Audit technique	configuration des systèmes, réseaux, applications	rapport de test d'intrusion
Audit de conformité	respect d'une réglementation précise	rapport PCI-DSS, RGPD
Audit de code	revue du code source	rapport SAST (analyse statique)
Audit physique	sécurité des locaux, contrôle d'accès	rapport d'inspection physique

L'**audit organisationnel** s'intéresse à ce que l'organisation a décidé et documenté (politiques de sécurité, procédures de gestion des incidents, gouvernance) ainsi qu'à la manière dont ces décisions sont réellement appliquées au quotidien. L'**audit technique**, à l'inverse, examine directement l'état réel des systèmes : configuration d'un pare-feu, version d'un serveur, présence d'une faille exploitable. Ces deux familles se complètent nécessairement : un système peut être « techniquement » bien configuré à un instant donné mais dépourvu de toute procédure formalisée de gestion des correctifs (le bon état actuel risque alors de se dégrader avec le temps, faute de processus), ou au contraire disposer de procédures très détaillées sur le papier mais jamais appliquées en pratique par les équipes opérationnelles. Ce module se concentre principalement sur ces deux premières familles — l'audit organisationnel (chapitre 2) et l'audit technique (chapitres 3 à 5) — tout en gardant à l'esprit que l'audit de conformité, l'audit de code et l'audit physique en sont des déclinaisons ou des compléments spécialisés.

4. Les acteurs et postures

Deux dimensions permettent de caractériser la posture adoptée pour un audit : la relation de l'auditeur à l'organisation auditée, et le niveau d'information dont il dispose au départ.

Selon la relation à l'organisation :

- **Auditeur interne** : appartient à l'organisation auditée (par exemple un service d'audit interne ou une équipe sécurité distincte des équipes d'exploitation). Il connaît bien le contexte, l'historique et les contraintes internes, ce qui accélère le travail, mais son indépendance reste relative — il évolue dans la même structure hiérarchique que les équipes auditées.
- **Auditeur externe** : société tierce spécialisée, mandatée pour l'occasion. Elle apporte une indépendance plus forte et un regard neuf, souvent exigé explicitement par les référentiels de certification (un audit de certification ISO 27001 ne peut d'ailleurs pas être réalisé par l'organisation elle-même).

Selon le niveau d'information initial (particulièrement pertinent pour l'audit technique) :

- **Boîte noire (*black box*)** : l'auditeur ne dispose d'aucune information préalable sur le système, exactement comme un attaquant externe au premier jour de sa reconnaissance. Cette posture est la plus réaliste vis-à-vis d'une menace externe, mais aussi la plus longue et la moins exhaustive dans un temps donné.
- **Boîte grise (*grey box*)** : l'auditeur dispose d'informations partielles (par exemple un compte utilisateur standard, une documentation d'architecture générale). Elle simule une menace interne, un partenaire malveillant, ou un attaquant externe ayant déjà obtenu un premier accès limité — un scénario très courant en pratique.
- **Boîte blanche (*white box*)** : l'auditeur a accès à toutes les informations disponibles (code source, schémas d'architecture détaillés, comptes administrateurs). Cette posture maximise la couverture et la profondeur de l'audit dans le temps imparti, au prix d'un scénario moins réaliste vis-à-vis d'un attaquant externe qui, lui, ne dispose pas de ces informations au départ.

Le choix de la posture n'est pas anodin : il conditionne directement ce que l'audit peut réellement démontrer, et doit donc être discuté et formalisé dès le cadrage.

5. Cadre légal et éthique

Tout audit technique — en particulier un test d'intrusion — implique, par nature, des actions qui, réalisées sans autorisation, seraient qualifiées d'accès et de maintien frauduleux dans un système de traitement automatisé de données au regard de la plupart des législations sur la cybercriminalité. La frontière entre un audit légitime et une intrusion illégale ne tient donc pas à la technique employée (les mêmes outils et les mêmes commandes peuvent être utilisés dans les deux cas), mais uniquement à l'existence d'une **autorisation préalable, explicite et écrite**.

Un audit ne peut être mené que dans le cadre d'un **mandat écrit** (parfois appelé lettre d'autorisation ou *rules of engagement*), qui précise notamment :

- le **périmètre exact autorisé** : adresses IP, noms de domaine, applications précises, plages horaires pendant lesquelles les tests peuvent être menés — toute cible en dehors de ce périmètre est strictement hors mandat, même si elle appartient techniquement à la même organisation ;
- les **techniques autorisées et interdites** : par exemple, les attaques par déni de service sont presque toujours exclues par défaut, car elles peuvent interrompre un service en production sans que cela apporte une information proportionnée au risque encouru ;
- les **responsables à contacter** en cas d'incident pendant l'audit, afin de pouvoir interrompre rapidement les tests si un système se comporte de façon anormale ;
- les **modalités de confidentialité et de destruction des données collectées** une fois l'audit terminé, car les preuves recueillies (identifiants extraits, données sensibles consultées) constituent elles-mêmes un risque si elles ne sont pas protégées puis détruites correctement.

Au-delà du cadre strictement légal, l'auditeur est tenu à une **déontologie professionnelle** : confidentialité absolue sur les constats avant la remise du rapport officiel, absence de conflit d'intérêt, et proportionnalité des moyens employés par rapport à l'objectif de démonstration (ne pas exploiter une vulnérabilité au-delà de ce qui est nécessaire pour en prouver l'existence et l'impact).

6. Le cycle de vie d'un audit

Un audit, qu'il soit organisationnel ou technique, suit généralement un cycle en plusieurs étapes qui structure l'ensemble de la démarche, du cadrage initial jusqu'au suivi de la remédiation :

1. **Cadrage** : définition précise du périmètre, des objectifs, des contraintes de temps et de moyens, et rédaction du mandat écrit qui encadre juridiquement l'ensemble de la mission.
2. **Collecte d'information / reconnaissance** : rassemblement de tout ce qui est nécessaire pour comprendre le système ou l'organisation auditée (documentation, entretiens, informations techniques publiquement accessibles).
3. **Analyse** (organisationnelle et/ou technique) : confrontation méthodique de ce qui a été observé aux exigences du référentiel choisi, à l'aide de grilles d'évaluation, de tests et d'outils appropriés.
4. **Identification et qualification des écarts/vulnérabilités** : chaque écart constaté est décrit précisément et évalué en termes de criticité, afin de pouvoir être priorisé.
5. **Rédaction du rapport** : mise en forme des constats, de leur criticité et des recommandations associées, dans un document structuré et adapté à ses différents lecteurs (voir chapitre 6).
6. **Restitution et plan d'action** : présentation des résultats aux parties prenantes (souvent en deux temps — synthèse pour la direction, détails pour les équipes techniques) et définition d'un plan de remédiation priorisé.
7. **Suivi** (contre-audit / vérification de la remédiation) : un audit isolé ne garantit rien dans la durée ; un suivi ultérieur permet de vérifier que les corrections annoncées ont bien été mises en œuvre et sont efficaces.

Ce cycle n'est pas purement linéaire dans la pratique : certaines phases (notamment la collecte d'information et l'analyse) s'enchaînent souvent de façon itérative, chaque nouvel élément découvert pouvant réorienter la recherche suivante.

À retenir

- Un audit combine une dimension organisationnelle et une dimension technique, et son indépendance vis-à-vis de l'entité auditée conditionne sa crédibilité.
- Les motivations d'un audit (conformité réglementaire, exigence contractuelle, gestion des risques, amélioration continue) déterminent son périmètre et son niveau d'exigence.
- Le mandat écrit est un prérequis non négociable à toute activité d'audit technique : sans autorisation explicite, les mêmes actions constituent une infraction.
- Le mode d'accès (black/grey/white box) détermine la couverture et le réalisme de l'audit, et doit être choisi en fonction du scénario de menace que l'on souhaite évaluer.
- Le cycle de vie d'un audit ne s'arrête pas au rapport : le suivi de la remédiation fait partie intégrante d'une démarche de sécurité mature.

Chapitre 2 — Audit organisationnel : normes et référentiels

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

ISO/IEC 27001 est une norme internationale certifiable qui définit les exigences d'un **Système de management de la sécurité de l'information (SMSI)**. Elle ne dit pas précisément *quelles* mesures techniques appliquer, mais *comment* une organisation doit piloter sa sécurité dans la durée : identifier ses risques, décider des mesures à mettre en œuvre, les appliquer, vérifier leur efficacité, et les améliorer en continu. C'est cette dimension de pilotage — et non une liste figée de bonnes pratiques — qui distingue un SMSI d'une simple accumulation de dispositifs de sécurité.

Cette logique de pilotage est structurée autour du cycle **PDCA** (*Plan-Do-Check-Act*), un modèle d'amélioration continue emprunté au management de la qualité :

- **Plan** : identifier les risques, définir la politique de sécurité et les objectifs.
- **Do** : mettre en œuvre les mesures de sécurité décidées (contrôles techniques et organisationnels).
- **Check** : vérifier l'efficacité des mesures (audits internes, indicateurs, revues de direction).
- **Act** : corriger les écarts constatés et améliorer le dispositif pour le cycle suivant.

ISO/IEC 27002, de son côté, n'est pas une norme certifiable en elle-même : c'est un **catalogue de mesures de sécurité** (bonnes pratiques), organisées en quatre thèmes (organisationnel, humain, physique, technologique), que l'ISO 27001 référence dans son Annexe A comme réservoir de contrôles possibles. En pratique, les deux normes fonctionnent en couple : l'ISO 27001 impose la démarche de management, et l'ISO 27002 fournit le détail opérationnel des mesures parmi lesquelles l'organisation sélectionne celles pertinentes pour son contexte, en le justifiant dans un document appelé Déclaration d'Applicabilité (SoA — *Statement of Applicability*).

Clauses principales de l'ISO 27001 (4 à 10)

Clause	Contenu
4	Contexte de l'organisation
5	Leadership (engagement de la direction, politique SSI)
6	Planification (appréciation et traitement des risques)
7	Support (ressources, compétences, sensibilisation)
8	Fonctionnement (mise en œuvre opérationnelle)
9	Évaluation des performances (audit interne, revue de direction)
10	Amélioration (non-conformités, actions correctives)

Un audit organisationnel vérifie, pour chaque clause, deux niveaux de preuve distincts et complémentaires : des **preuves documentaires** (l'existence d'une politique, d'une procédure écrite, d'un registre des risques) et des **preuves de mise en œuvre effective** (entretiens avec les

2. Autres référentiels usuels

Au-delà de la famille ISO 27001/27002, plusieurs autres référentiels sont fréquemment mobilisés dans un audit organisationnel, chacun avec un objet et un usage propres. Il est essentiel de savoir les distinguer, car un audit mal cadré risque de mélanger des exigences qui ne relèvent pas du même registre (certification volontaire, obligation contractuelle, obligation légale).

Référentiel	Portée	Usage typique
PCI-DSS	données de cartes bancaires	commerçants et prestataires de paiement
NIST Cybersecurity Framework	gestion du risque cyber (Identify, Protect, Detect, Respond, Recover)	référence internationale, souvent utilisée hors certification
ANSSI (guides et référentiels)	administrations et OIV en France, largement repris en Afrique francophone	hygiène informatique, PASSI (prestataires d'audit)
RGPD	protection des données personnelles	conformité juridique, articulé avec la sécurité technique
CIS Controls	mesures techniques prioritées	audit technique et durcissement (<i>hardening</i>)

Le **PCI-DSS** (*Payment Card Industry Data Security Standard*) est imposé de fait par les réseaux de cartes bancaires à toute entité qui traite, stocke ou transmet des données de porteurs de cartes ; contrairement à l'ISO 27001, ce n'est pas une démarche volontaire mais une exigence contractuelle incontournable pour opérer dans ce secteur. Le **NIST Cybersecurity Framework**, développé par l'agence américaine de normalisation, propose une grille de lecture en cinq fonctions (Identifier, Protéger, Détecter, Répondre, Récupérer) largement reprise à l'international même en dehors de tout contexte réglementaire américain, notamment parce qu'elle offre un vocabulaire commun simple pour discuter de maturité cyber avec une direction non technique. Le **RGPD**, quant à lui, n'est pas à proprement parler un référentiel de sécurité mais un texte réglementaire sur la protection des données personnelles : il impose des obligations de sécurité (article 32) qui recoupent largement les bonnes pratiques ISO 27002, ce qui explique qu'un audit de conformité RGPD s'appuie souvent sur les mêmes grilles qu'un audit organisationnel classique. Enfin, les **CIS Controls** se distinguent des référentiels précédents par leur nature très opérationnelle : ce sont des mesures techniques concrètes et prioritées (par exemple « tenir un inventaire des équipements autorisés »), qui font le pont entre l'audit organisationnel et l'audit technique abordé au chapitre suivant.

3. Méthodologie d'un audit organisationnel

Un audit organisationnel suit une progression logique, chaque étape s'appuyant sur les résultats de la précédente pour affiner progressivement le constat :

1. **Revue documentaire** : l'auditeur commence par collecter et analyser l'ensemble des documents formels — politiques de sécurité, chartes utilisateurs, procédures de gestion des incidents, plan de continuité d'activité (PCA) et plan de reprise d'activité (PRA). Cette étape permet de comprendre ce que l'organisation *dit* faire, mais ne suffit jamais à elle seule.
2. **Entretiens** avec les parties prenantes (RSSI, DSI, ressources humaines, responsables métiers) : ils permettent de confronter le discours documentaire à la perception réelle des équipes, et souvent de révéler des pratiques informelles absentes de toute procédure écrite.
3. **Échantillonnage** : plutôt que de contrôler l'exhaustivité des tickets, comptes utilisateurs ou journaux (souvent impossible dans le temps imparti), l'auditeur sélectionne un échantillon représentatif et en vérifie la conformité en détail — une méthode empruntée à l'audit comptable, adaptée au contexte de la sécurité de l'information.
4. **Analyse d'écart (*gap analysis*)** : chaque exigence du référentiel choisi est confrontée à l'état constaté (documentaire, entretiens, échantillons), pour produire une liste structurée d'écarts, qu'ils soient mineurs (formulation imprécise d'une procédure) ou majeurs (absence totale de dispositif).
5. **Notation de maturité** : les organisations plus matures vont au-delà d'un simple constat binaire conforme/non conforme, et notent chaque domaine sur une échelle de maturité (souvent inspirée du modèle CMMI adapté : 0 = inexistant, 1 = initial/ad hoc, 2 = reproductible mais informel, 3 = défini et documenté, 4 = maîtrisé et mesuré, 5 = optimisé en amélioration continue). Cette notation permet de suivre une trajectoire dans le temps, plutôt qu'une simple photographie instantanée.

4. Exemple de grille d'évaluation simplifiée

Voici un extrait illustratif d'une grille d'évaluation telle qu'un auditeur pourrait la constituer au fil de ses entretiens et de sa revue documentaire. Chaque ligne relie une exigence du référentiel à un constat factuel et à une note de maturité, ce qui permet ensuite de prioriser les actions correctives (chapitre 6) :

Domaine	Exigence	Constat	Niveau de maturité (0-5)
Gestion des accès	Revue périodique des droits	Aucune revue formalisée depuis 18 mois	1
Gestion des correctifs	Politique de patch management documentée	Politique existe, appliquée à 60 % des serveurs	2
Sensibilisation	Formation annuelle du personnel	Formation dispensée à l'embauche uniquement	2

Le constat « aucune revue formalisée depuis 18 mois » illustre bien la distinction entre existence d'une procédure et efficacité réelle : il est probable qu'une politique de gestion des accès existe sur le papier, mais que son application se soit progressivement arrêtée faute de suivi — un scénario extrêmement fréquent en pratique, et une des raisons pour lesquelles la revue documentaire seule ne suffit jamais à qualifier une maturité organisationnelle.

5. Risques d'un audit organisationnel mal mené

Un audit organisationnel mal conduit peut produire un rapport qui rassure à tort, ce qui est en réalité pire que l'absence d'audit — l'organisation croit être couverte alors qu'elle ne l'est pas. Trois écueils reviennent fréquemment :

- **Se limiter à la lecture des documents sans vérifier l'application réelle** (« conformité de façade ») : un auditeur pressé ou peu rigoureux peut se contenter de vérifier que les documents requis existent formellement, sans jamais interroger les équipes opérationnelles ni échantillonner de preuves concrètes. Le rapport final semble alors positif, mais ne reflète pas la réalité du terrain.
- **Confondre existence d'une procédure et efficacité de la procédure** : une procédure de gestion des incidents peut exister, être bien rédigée, et pourtant n'avoir jamais été testée ni suivie lors d'un incident réel — ce qui la rend en pratique inefficace au moment où elle serait nécessaire.
- **Absence de priorisation** : toutes les non-conformités identifiées ne présentent pas le même niveau de risque. Un rapport qui liste vingt écarts sans hiérarchie claire entre eux (un formulaire mal archivé au même niveau qu'une absence totale de gestion des accès à privilèges) noie l'essentiel dans l'accessoire et complique la prise de décision par la direction.

À retenir

- L'audit organisationnel évalue des processus et de la documentation, pas uniquement des configurations techniques, et repose sur deux types de preuves complémentaires : documentaires et de mise en œuvre effective.
- ISO 27001/27002 est le référentiel le plus largement utilisé, avec une logique de pilotage continu (PDCA) plutôt qu'une simple liste de contrôles ; il se combine souvent avec des référentiels sectoriels (PCI-DSS) ou nationaux (ANSSI).
- La méthodologie type (revue documentaire, entretiens, échantillonnage, analyse d'écart, notation de maturité) permet de dépasser une lecture purement déclarative de la sécurité de l'organisation.
- Une non-conformité doit toujours être qualifiée par un niveau de criticité et une recommandation actionnable, sous peine de produire un rapport peu exploitable par la direction.

Chapitre 3 — Audit technique : méthodologie des tests d'intrusion

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le PTES (Penetration Testing Execution Standard)

Le **PTES** (*Penetration Testing Execution Standard*) est l'un des référentiels méthodologiques les plus utilisés pour cadrer et dérouler un test d'intrusion. Son intérêt principal est de fournir un vocabulaire et une structure communs entre auditeurs, ce qui facilite la comparaison de prestations entre différents prestataires et la relecture d'un rapport par un client qui ne maîtrise pas nécessairement le détail technique. Il structure un test d'intrusion en sept phases, qui s'enchaînent logiquement mais peuvent, en pratique, se recouper ou se répéter au fil des découvertes :

1. **Pre-engagement** : cadrage, périmètre, mandat, règles d'engagement — cette phase est directement liée au cadre légal vu au chapitre 1 : rien de technique ne commence avant que le mandat écrit ne soit signé.
2. **Intelligence gathering** (reconnaissance) : collecte passive et active d'informations sur la cible, afin de construire une vision aussi complète que possible de sa surface d'attaque avant toute tentative d'exploitation.
3. **Threat modeling** : identification des actifs critiques (données sensibles, systèmes vitaux) et des scénarios d'attaque plausibles compte tenu du profil de la cible — cette étape oriente les efforts vers ce qui compte réellement plutôt que de tester tout, indifféremment, avec la même intensité.
4. **Vulnerability analysis** : identification des failles, par combinaison de scans automatisés et d'analyse manuelle, pour dresser la liste des points d'entrée potentiels.
5. **Exploitation** : tentative d'exploitation contrôlée des vulnérabilités identifiées, dans le but de démontrer leur impact réel plutôt que de se contenter d'une simple hypothèse théorique.
6. **Post-exploitation** : une fois un accès obtenu, évaluation de l'impact réel qu'un attaquant pourrait en tirer (élévation de privilèges, mouvement latéral ou « pivot » vers d'autres systèmes, simulation d'exfiltration de données) — c'est souvent cette phase qui transforme une vulnérabilité isolée en un scénario d'incident crédible pour la direction.
7. **Reporting** : rédaction du rapport, qui documente l'ensemble de la démarche et de ses résultats (voir chapitre 6 pour la structure détaillée d'un rapport).

2. Reconnaissance passive vs active

La phase de reconnaissance se subdivise en deux approches complémentaires, dont la distinction est fondamentale car elle conditionne le risque de détection par la cible.

Type	Exemples de techniques	Délectable par la cible ?
Passive	OSINT (WHOIS, réseaux sociaux, moteurs de recherche, theHarvester), consultation de code source public	non
Active	scan de ports, résolution DNS directe, requêtes HTTP vers la cible	oui, potentiellement journalisé

La **reconnaissance passive** consiste à rassembler des informations sur la cible sans jamais interagir directement avec ses systèmes : consultation de sources publiques (registres WHOIS, réseaux sociaux, dépôts de code source publics, moteurs de recherche), ce que l'on regroupe sous le terme d'**OSINT** (*Open Source Intelligence*). Comme aucune requête n'est envoyée directement à la cible, cette activité est par définition indétectable par elle. Elle permet néanmoins de récolter des informations précieuses : noms de domaine associés, adresses e-mail de contact, technologies mentionnées dans des offres d'emploi, employés identifiables sur les réseaux professionnels — autant d'éléments qui alimentent ensuite les phases suivantes.

La **reconnaissance active**, à l'inverse, implique un contact direct avec l'infrastructure de la cible (scan de ports, résolution DNS, requêtes HTTP). Ces actions laissent des traces dans les journaux de la cible et peuvent déclencher des alertes de détection — ce qui en fait, incidemment, un excellent test de la capacité de détection de l'organisation, même lorsque ce n'est pas l'objectif premier de l'exercice.

3. Scan et énumération

Une fois la reconnaissance initiale effectuée, l'auditeur cherche à préciser concrètement quels services sont exposés et comment ils sont configurés.

- **Scan de ports** : identifie les services exposés sur une machine cible, en distinguant les ports ouverts (un service y écoute), fermés (aucun service, mais la machine répond) et filtrés (aucune réponse, généralement à cause d'un pare-feu). L'outil de référence pour cette tâche est `nmap`.
- **Fingerprinting** : au-delà de la simple ouverture d'un port, cette technique cherche à identifier précisément la version du service et du système d'exploitation sous-jacent (option `-sV -O` de `nmap`), une information indispensable pour rechercher ensuite des vulnérabilités connues associées à cette version précise.
- **Énumération** : consiste à lister méthodiquement les ressources exposées par un service identifié — partages réseau accessibles, comptes utilisateurs valides, répertoires et fichiers d'une application web. C'est une étape souvent sous-estimée alors qu'elle révèle fréquemment des informations sensibles exposées par erreur (fichiers de sauvegarde oubliés, répertoires d'administration non protégés).

```
nmap -sV -sC -p- 192.168.1.10
```

Dans cet exemple, `-sV` active la détection de version des services, `-sC` exécute un ensemble de scripts de reconnaissance standard fournis avec `nmap`, et `-p-` étend le scan à l'ensemble des 65 535 ports TCP plutôt qu'aux seuls ports les plus courants — une pratique recommandée en audit, car limiter le scan aux ports par défaut laisse échapper les services volontairement déplacés sur un port non standard.

4. Analyse de vulnérabilités

Une fois les services et leurs versions identifiés, l'étape suivante consiste à déterminer si des vulnérabilités connues leur sont associées. Des outils automatisés (scanners) confrontent systématiquement les versions relevées à des bases de vulnérabilités publiques référencées sous forme de **CVE** (*Common Vulnerabilities and Exposures*) :

- **Nessus, OpenVAS** : scanners généralistes orientés réseau et systèmes, capables de tester un grand nombre de services et de configurations en une seule campagne de scan.
- **Nikto, OWASP ZAP, Burp Suite** : scanners spécifiquement orientés applications web, qui recherchent des configurations dangereuses ou des vulnérabilités typiques des applications HTTP (en-têtes manquants, formulaires vulnérables, points d'entrée non protégés).

Un scan automatisé, aussi puissant soit-il, présente des limites structurelles qu'il est essentiel de connaître avant de s'y fier aveuglément. Il produit des **faux positifs** — une vulnérabilité signalée sur la base d'une simple correspondance de version, alors qu'elle a en réalité été corrigée par un correctif appliqué séparément (*backport*) sans changement de numéro de version visible. Il peut aussi laisser passer des **faux négatifs** : des vulnérabilités purement logiques (par exemple un contrôle d'accès mal conçu métier par métier) ne correspondent à aucune signature connue et ne peuvent être détectées que par une analyse manuelle attentive. C'est pourquoi un scan automatisé, quel que soit sa qualité, ne remplace jamais le jugement et l'expérience d'un auditeur humain — il en constitue tout au plus un point de départ efficace.

5. Exploitation contrôlée

L'exploitation est l'étape qui distingue véritablement un test d'intrusion d'un simple scan de vulnérabilités : elle vise à démontrer concrètement l'impact réel d'une faille identifiée, sous la forme d'une preuve de concept, sans jamais dégrader le service ni les données de production visées. Trois principes encadrent cette démarche :

- **Privilégier les preuves non destructives** : lorsque plusieurs façons existent de démontrer une vulnérabilité, l'auditeur choisit toujours la moins risquée pour la cible — par exemple lire le contenu d'un fichier témoin déposé volontairement à cet effet plutôt que supprimer ou modifier des données réelles, ce qui suffit largement à prouver l'existence et la gravité de la faille.
- **Documenter précisément chaque action entreprise** : horodatage, commande exacte utilisée, résultat obtenu — cette traçabilité minutieuse sert à la fois de preuve pour le rapport final et de garde-fou en cas de contestation ultérieure sur ce qui a réellement été fait pendant l'audit.
- **S'arrêter strictement au périmètre autorisé par le mandat**, même lorsqu'une extension du périmètre semble techniquement possible et tentante — franchir cette limite, même dans une intention purement démonstrative, transformerait un audit légitime en action non autorisée.

6. Notation de la criticité (CVSS)

Le score **CVSS** (*Common Vulnerability Scoring System*) est un standard largement adopté pour qualifier de façon objective et reproductible la gravité d'une vulnérabilité, à partir de plusieurs métriques combinées : le vecteur d'attaque (locale, réseau adjacent, réseau ouvert), la complexité de l'exploitation, les privilèges requis avant de pouvoir exploiter la faille, l'interaction utilisateur nécessaire, et l'impact sur les trois piliers de la sécurité de l'information — confidentialité, intégrité et disponibilité. Le score final est exprimé sur une échelle de 0 à 10, généralement traduite en catégories qualitatives (faible, moyen, élevé, critique).

L'intérêt principal du CVSS est de rendre les comparaisons possibles entre vulnérabilités hétérogènes, plutôt que de reposer uniquement sur l'appréciation subjective d'un auditeur. Un score CVSS de 9.8 sur 10 (typiquement une exécution de code à distance sans authentification préalable requise) impose une remédiation immédiate, alors qu'un score de 3.1 (par exemple une fuite d'information mineure nécessitant un accès local déjà privilégié) peut généralement être traité dans un cycle de correctifs standard. Il faut néanmoins garder à l'esprit que le CVSS mesure une gravité technique intrinsèque, indépendamment du contexte métier de la cible — un point approfondi au chapitre 6, où la criticité technique brute est recontextualisée avec l'impact métier réel.

7. Limites et complémentarité avec l'audit organisationnel

Un test d'intrusion technique, aussi rigoureux soit-il, donne une **photographie à un instant t** d'un périmètre limité, défini par le mandat. Il ne dit rien, en lui-même, sur la capacité de l'organisation à *détecter* une attaque en cours, ni sur sa capacité à y *répondre* efficacement une fois détectée — deux dimensions pourtant essentielles de la sécurité opérationnelle. Un test d'intrusion classique se termine généralement dès qu'un accès significatif est démontré, sans chercher systématiquement à savoir si les équipes de supervision l'ont remarqué.

C'est précisément la différence avec un exercice de type **Red Team**, plus large et généralement plus long dans le temps, dont l'objectif explicite inclut de tester également la détection et la réaction de l'organisation (« Blue Team »), et pas seulement la capacité d'intrusion elle-même. Cette complémentarité illustre bien pourquoi l'audit technique seul ne suffit jamais : il doit être articulé avec l'audit organisationnel (chapitre 2), qui évalue justement les processus de détection, de réponse à incident et de gouvernance qu'un test d'intrusion ponctuel ne peut pas mesurer directement.

À retenir

- Le PTES structure la démarche en sept phases, de la reconnaissance au rapport, en s'appuyant sur un vocabulaire commun facilitant la comparaison entre prestations.
- La reconnaissance se décompose en une composante passive (indétectable) et active (potentiellement journalisée), et l'analyse de vulnérabilités combine scan automatisé et jugement manuel.
- L'automatisation (scanners) accélère l'analyse mais ne remplace pas le jugement de l'auditeur, en raison des faux positifs et des faux négatifs qu'elle produit structurellement.
- L'exploitation doit rester non destructive, documentée et strictement bornée au périmètre du mandat.
- Le score CVSS objective la priorisation des vulnérabilités identifiées, mais doit être recontextualisé au regard de l'impact métier réel.
- Un test d'intrusion technique ne mesure pas la capacité de détection et de réponse de l'organisation ; seul un exercice de type Red Team ou l'audit organisationnel permet d'évaluer cette dimension.

Chapitre 4 — Audit des infrastructures réseau et systèmes

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Audit de l'architecture réseau

L'architecture réseau constitue la première ligne de défense d'un système d'information, et son audit vise à vérifier qu'elle limite effectivement la propagation d'une compromission plutôt que de simplement filtrer le trafic entrant.

- **Segmentation** : il s'agit de vérifier l'existence de zones réseau distinctes (DMZ pour les services exposés à Internet, réseau interne pour les postes de travail, réseau d'administration dédié aux équipements critiques) et que le filtrage entre ces zones suit le **principe du moindre privilège** — c'est-à-dire que seuls les flux strictement nécessaires au fonctionnement du système sont autorisés à traverser une frontière de zone. Une segmentation absente ou mal conçue signifie qu'un attaquant ayant compromis une seule machine peu sensible (par exemple un poste utilisateur) peut ensuite se déplacer librement vers des systèmes bien plus critiques (serveurs de base de données, contrôleurs de domaine).
- **Flux autorisés** : l'auditeur procède à une revue systématique des règles de pare-feu. Une règle de type `ANY-ANY` (tout autorisé, quelle que soit la source, la destination ou le port), qu'elle soit en entrée ou en sortie, est presque toujours une non-conformité à documenter : elle indique généralement que la règle a été créée par facilité pour résoudre un problème ponctuel, sans jamais avoir été resserrée par la suite.
- **Points d'entrée exposés** : il s'agit de recenser exhaustivement tout ce qui est accessible depuis Internet — accès VPN, portails d'administration, services qui devaient rester internes mais ont été exposés par erreur de configuration. Ce recensement s'appuie souvent sur des outils de cartographie externe (scan depuis l'extérieur du réseau) car la documentation interne de l'organisation ne reflète pas toujours fidèlement la réalité de ce qui est réellement exposé.

2. Audit des équipements réseau

Les équipements réseau (routeurs, commutateurs, imprimantes réseau, caméras IP et autres objets connectés) constituent une catégorie d'actifs souvent moins surveillée que les serveurs applicatifs, alors qu'ils présentent des faiblesses récurrentes et bien documentées :

- **Mots de passe par défaut non changés** : de nombreux équipements sont livrés avec des identifiants d'administration standard, documentés publiquement par le fabricant ; laisser ces identifiants inchangés équivaut, en pratique, à ne pas protéger l'équipement du tout.
- **Protocoles non chiffrés encore actifs** : Telnet pour l'administration à distance, interfaces d'administration en HTTP simple plutôt qu'en HTTPS, SNMP en version 1 ou 2 utilisant la chaîne de communauté par défaut `public` (qui donne un accès en lecture, voire en écriture selon la configuration, sans authentification réelle). Ces protocoles historiques transmettent les informations, y compris les identifiants, en clair sur le réseau.
- **Firmware obsolète et absence de politique de mise à jour** : à la différence d'un serveur, un équipement réseau est souvent installé une fois puis oublié pendant des années, sans processus formel de mise à jour de son firmware, alors que des vulnérabilités y sont régulièrement découvertes et corrigées par les fabricants.

3. Audit des systèmes d'exploitation (hardening)

Le **durcissement** (*hardening*) d'un système d'exploitation consiste à réduire sa surface d'attaque en désactivant tout ce qui n'est pas strictement nécessaire et en renforçant la configuration des éléments conservés. Les référentiels de durcissement les plus utilisés en pratique sont les **CIS Benchmarks** (*Center for Internet Security*), qui fournissent des points de contrôle précis et mesurables par famille de systèmes (Windows Server, distributions Linux, équipements réseau, conteneurs). Un audit de durcissement s'organise généralement autour des domaines suivants :

Domaine	Points de contrôle
Comptes et authentification	comptes par défaut désactivés, politique de mots de passe, MFA pour les comptes à privilèges
Services	services inutiles désactivés (réduction de la surface d'attaque)
Correctifs	politique de patch management, délai moyen d'application des correctifs critiques
Journalisation	logs activés, centralisés, conservés une durée suffisante
Droits	principe du moindre privilège, séparation des comptes admin/utilisateur

Le domaine **comptes et authentification** vise à s'assurer que les comptes livrés par défaut avec le système (souvent avec des droits élevés) sont désactivés ou renommés, qu'une politique de mots de passe robuste est appliquée techniquement (et pas seulement recommandée par écrit), et surtout que l'authentification multi-facteurs (MFA) protège les comptes disposant de privilèges d'administration — ces comptes étant la cible privilégiée de tout attaquant cherchant à étendre son contrôle. Le domaine **services** repose sur un principe simple : chaque service actif sur une machine constitue un point d'entrée potentiel supplémentaire ; désactiver ceux qui ne sont pas utilisés réduit mécaniquement la surface exposée, sans nécessiter de mesure de sécurité additionnelle. Le domaine **correctifs** ne se limite pas à vérifier l'existence d'une politique écrite : il mesure concrètement le délai moyen entre la publication d'un correctif critique et son application effective sur le parc, un indicateur souvent bien plus révélateur que la simple existence de la politique. Enfin, la **journalisation** conditionne la capacité de l'organisation à détecter une compromission et à en reconstituer le déroulement a posteriori — des journaux non centralisés ou conservés trop peu de temps rendent toute investigation ultérieure très difficile, voire impossible.

4. Cas particulier : audit d'un domaine Active Directory

L'annuaire **Active Directory** est une cible privilégiée dans la quasi-totalité des environnements Windows d'entreprise, car il centralise l'authentification et l'autorisation de l'ensemble du parc : compromettre le domaine revient souvent à compromettre l'intégralité du système d'information. Les points d'audit les plus fréquemment examinés sont :

- **comptes avec mot de passe n'expirant jamais** ou marqués `PASSWD_NOTREQD` (mot de passe non requis), qui représentent des cibles faciles pour une attaque par force brute ou par devinette, d'autant plus dangereuses qu'elles ne sont soumises à aucune politique de renouvellement ;
- **délégation Kerberos non contrainte mal maîtrisée**, un mécanisme technique avancé qui, mal configuré, permet à un attaquant ayant compromis un serveur disposant de cette délégation d'usurper l'identité de n'importe quel autre utilisateur du domaine qui s'y connecte, y compris des administrateurs ;
- **groupes à privilèges surdimensionnés** (typiquement le groupe `Domain Admins`) : plus ce groupe compte de membres, plus la probabilité qu'un compte compromis en fasse partie augmente, et plus la surface d'attaque effective sur les privilèges les plus élevés du domaine s'élargit ;
- **absence de séparation entre comptes d'administration et comptes utilisateurs quotidiens** : un administrateur qui utilise le même compte pour consulter ses e-mails et pour administrer les serveurs expose ses privilèges élevés à toute compromission de son poste de travail habituel, un vecteur d'attaque très courant ;
- **outils dédiés d'audit** : `BloodHound` permet de cartographier automatiquement les chemins d'attaque possibles au sein d'un domaine Active Directory, en révélant des combinaisons de permissions qu'aucun administrateur n'aurait identifiées manuellement ; `PingCastle` produit un score de maturité synthétique du domaine, comparable à un tableau de bord de risque, utile pour communiquer rapidement l'état de sécurité de l'annuaire à une direction non technique.

5. Audit du Cloud et des environnements virtualisés

Le déplacement croissant des infrastructures vers le Cloud déplace également le périmètre et la nature de l'audit d'infrastructure, sans en réduire l'importance :

- **Vérification des configurations de stockage** : les services de stockage objet (buckets S3 ou équivalents chez d'autres fournisseurs) sont fréquemment configurés par erreur en accès public, exposant des données parfois sensibles à quiconque en devine ou découvre l'adresse — un défaut de configuration, et non une vulnérabilité logicielle, ce qui rend ce type de contrôle particulièrement important en audit Cloud.
- **Gestion des identités et accès (IAM)** : l'audit vérifie que les clés d'API sont régulièrement renouvelées (« tournées »), et que les permissions accordées à chaque identité (utilisateur, service, application) respectent le principe du moindre privilège plutôt que des permissions larges accordées par facilité de mise en œuvre initiale.
- **Isolation entre locataires (tenants)** dans un environnement mutualisé : dans les environnements Cloud publics où plusieurs clients partagent la même infrastructure physique sous-jacente, l'audit s'assure que les mécanismes d'isolation logique empêchent effectivement qu'un client puisse accéder, même accidentellement, aux ressources d'un autre.
- **Journalisation et alerte activées et supervisées** : les fournisseurs Cloud proposent généralement des services de journalisation des actions administratives (CloudTrail ou équivalent) ; leur simple activation ne suffit pas — encore faut-il que ces journaux soient effectivement supervisés et donnent lieu à des alertes exploitées, sans quoi ils ne servent qu'à une investigation a posteriori, une fois l'incident déjà survenu.

6. Restitution d'un audit d'infrastructure

Pour rester exploitable par des publics différents (équipes techniques et direction), chaque constat technique issu d'un audit d'infrastructure gagne à être formulé selon une structure homogène en quatre éléments : l'**observation** (le fait précisément constaté, accompagné d'une preuve reproductible), le **risque** (ce que cette observation permet concrètement à un attaquant de faire), la **recommandation** (l'action corrective à mener, si possible priorisée et estimée en effort) et la **référence** (le point du référentiel qui justifie le constat — CIS Benchmark, clause ISO 27002, identifiant CVE le cas échéant). Cette structure systématique, reprise en détail au chapitre 6, garantit qu'un constat technique reste toujours relié à un référentiel reconnu plutôt qu'à l'appréciation isolée d'un auditeur.

À retenir

- L'audit d'infrastructure combine architecture réseau, configuration système et, de plus en plus, environnements Cloud et Active Directory, chacun avec ses points de contrôle spécifiques.
- Les référentiels de durcissement (CIS Benchmarks) fournissent des points de contrôle réutilisables et mesurables, plus fiables qu'une simple appréciation qualitative.
- Active Directory reste une cible privilégiée en environnement Windows d'entreprise, avec des outils d'audit dédiés (BloodHound, PingCastle) qui automatisent la découverte de chemins d'attaque.
- Le Cloud déplace le périmètre de l'audit (configuration, IAM, isolation, journalisation) sans en réduire l'importance ni la rigueur attendue.
- Chaque constat doit relier un fait technique à un risque métier compréhensible par la direction, via une structure homogène : observation, risque, recommandation, référence.

Chapitre 5 — Audit des applications et du code

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi auditer les applications spécifiquement

Les applications web et métier constituent, dans la grande majorité des organisations modernes, la **surface d'attaque la plus exposée et la plus évolutive**. Contrairement à un équipement réseau qui reste configuré de façon relativement stable dans le temps, une application fait l'objet de déploiements fréquents (parfois plusieurs fois par jour dans une démarche DevOps), s'appuie sur du code sur mesure écrit par des équipes internes, et intègre de nombreuses dépendances tierces dont la sécurité échappe en partie à l'organisation elle-même. Cette combinaison — évolution rapide, code spécifique difficile à généraliser d'une application à l'autre, dépendances externes — rend l'audit applicatif à la fois plus délicat et plus nécessaire que l'audit d'une infrastructure statique.

Ce chapitre a une vocation volontairement introductive : il pose les bases méthodologiques de l'audit applicatif (référentiels, approches, méthodologie générale) et prépare directement le cours *Sécurité des applications*, qui approfondira chaque catégorie de vulnérabilité (injection, XSS, IDOR, etc.) avec le niveau de détail technique nécessaire pour les comprendre et les exploiter en profondeur.

2. OWASP Top 10 comme grille d'audit

L'**OWASP** (*Open Web Application Security Project*) est une organisation à but non lucratif qui produit, entre autres travaux, le classement **OWASP Top 10** : une liste des dix catégories de vulnérabilités applicatives jugées les plus critiques, régulièrement mise à jour pour refléter l'évolution des menaces. Ce classement n'est pas une liste exhaustive de toutes les vulnérabilités possibles, mais une **checklist minimale** largement reconnue dans l'industrie, qui sert de point de départ à tout audit applicatif digne de ce nom :

1. Contrôle d'accès défaillant (*broken access control*)
2. Défaillances cryptographiques
3. Injection (SQL, commande, etc.)
4. Conception non sécurisée (*insecure design*)
5. Mauvaise configuration de sécurité
6. Composants vulnérables et obsolètes
7. Défaillances d'identification et d'authentification
8. Défaillances d'intégrité des données et logiciels
9. Journalisation et surveillance insuffisantes
10. Falsification de requête côté serveur (SSRF)

Il est utile de noter que ce classement combine des catégories très techniques (l'injection, par exemple, se manifeste par des lignes de code précises et identifiables) et des catégories plus transverses, presque organisationnelles (la journalisation et surveillance insuffisantes rejoint directement les préoccupations vues en audit organisationnel au chapitre 2, et la conception non sécurisée renvoie à des choix d'architecture faits bien en amont de l'écriture du code). Un audit applicatif efficace ne se limite donc pas à chercher des lignes de code fautives : il interroge aussi les choix de conception et les pratiques de l'équipe de développement.

3. Approches d'audit applicatif

Plusieurs approches techniques permettent d'auditer une application, chacune avec sa portée, ses forces et ses limites propres. Une démarche mature ne choisit pas une seule approche mais les combine, en les positionnant à des moments différents du cycle de développement :

Approche	Description	Outils typiques
DAST (Dynamic Application Security Testing)	tests en boîte noire sur l'application en fonctionnement	OWASP ZAP, Burp Suite
SAST (Static Application Security Testing)	analyse du code source sans exécution	SonarQube, Semgrep, Bandit (Python), Cppcheck (C/C++)
SCA (Software Composition Analysis)	analyse des dépendances tierces et de leurs vulnérabilités connues	OWASP Dependency-Check, <code>npm audit</code>
Revue manuelle de code	lecture experte, notamment pour la logique métier	—

Le **DAST** teste l'application « de l'extérieur », exactement comme le ferait un attaquant qui n'a pas accès au code source : il envoie des requêtes à l'application en fonctionnement et observe ses réponses pour détecter un comportement anormal (par exemple une erreur SQL révélée dans une page d'erreur). Le **SAST**, à l'inverse, analyse directement le code source sans jamais exécuter l'application, ce qui lui permet de détecter des failles potentielles avant même la mise en production, mais au prix d'un taux de faux positifs souvent plus élevé (le SAST signale un usage de fonction dangereuse même lorsque le contexte rend l'exploitation impossible en pratique). Le **SCA** répond à un problème spécifique et de plus en plus critique : la quasi-totalité des applications modernes s'appuient sur un grand nombre de bibliothèques tierces, dont certaines peuvent contenir des vulnérabilités connues (référéncées par des CVE) que l'équipe de développement n'a pas nécessairement suivies — le SCA automatise cette veille en confrontant la liste des dépendances utilisées aux bases de vulnérabilités publiques. Enfin, la **revue manuelle de code** reste irremplaçable pour évaluer la logique métier de l'application : aucun outil automatisé ne peut, par exemple, déterminer si la règle « un utilisateur ne peut valider une commande que si son solde est suffisant » est correctement implémentée dans tous les cas, y compris les cas limites.

Une démarche mature combine ces approches en les positionnant différemment dans le temps : le SAST et le SCA s'intègrent naturellement au pipeline CI/CD pour un contrôle continu et automatique à chaque modification du code, tandis que le DAST et la revue manuelle, plus coûteux en temps humain, interviennent plutôt à intervalles réguliers ou avant une mise en production majeure.

4. Méthodologie d'un pentest applicatif

Un test d'intrusion applicatif suit une progression logique, qui va de la compréhension globale de l'application vers des tests de plus en plus ciblés :

1. **Cartographie** de l'application : recensement systématique des pages, fonctionnalités, rôles utilisateurs et points d'entrée de données (formulaires, paramètres d'URL, en-têtes HTTP, API exposées) — cette étape construit la carte complète sur laquelle s'appuieront tous les tests suivants.
2. **Tests d'authentification et de gestion de session** : robustesse de la politique de mot de passe, comportement en cas de tentatives répétées (protection contre le *brute force*), expiration correcte des sessions après déconnexion ou inactivité.
3. **Tests de contrôle d'accès** : la question centrale est de vérifier si un utilisateur standard peut accéder à des ressources appartenant à un autre utilisateur, voire à un administrateur, simplement en modifiant un identifiant dans l'URL ou dans une requête (une faille connue sous le nom d'**IDOR**, *Insecure Direct Object Reference*) — un exemple typique consiste à changer `/commandes/42` en `/commandes/43` et observer si l'application vérifie réellement que la commande 43 appartient bien à l'utilisateur connecté.
4. **Tests d'injection** (SQL, commande système, LDAP) sur chaque point de saisie de l'application : chaque champ où l'utilisateur peut fournir une donnée est un point d'entrée potentiel où une entrée malveillante pourrait être interprétée comme une instruction par le système sous-jacent plutôt que comme une simple donnée.
5. **Tests côté client** : XSS (*Cross-Site Scripting*) stocké ou réfléchi, qui permet d'injecter du code exécuté dans le navigateur d'autres utilisateurs, et CSRF (*Cross-Site Request Forgery*), qui exploite la confiance d'un site envers le navigateur authentifié d'un utilisateur pour lui faire exécuter une action à son insu.
6. **Tests de configuration** : présence des en-têtes de sécurité HTTP recommandés, gestion propre des erreurs (une page d'erreur qui révèle la structure interne du code ou de la base de données facilite considérablement le travail d'un attaquant), absence d'informations sensibles exposées par erreur (clés d'API, commentaires de débogage laissés dans le code source livré au client).

5. Audit de code : exemple de grille (langage C, en lien avec le module Algorithmique)

Certaines catégories de vulnérabilités sont spécifiques à des langages de bas niveau comme le C, où la gestion manuelle de la mémoire ouvre des risques qui n'existent tout simplement pas dans des langages gérant la mémoire automatiquement. Cette grille illustre les points de contrôle les plus classiques d'un audit de code C, en écho direct au module Algorithmique :

Point de contrôle	Risque associé
Utilisation de <code>strcpy</code> , <code>gets</code> , <code>sprintf</code> sans borne	débordement de tampon
Absence de vérification du retour de <code>malloc</code>	déréférencement de pointeur nul
<code>free</code> sans remise à <code>NULL</code>	use-after-free
Chaîne de format non constante passée à <code>printf</code>	vulnérabilité format string
Calculs de taille non vérifiés (multiplication avant <code>malloc</code>)	débordement d'entier

L'usage de fonctions comme `strcpy` ou `gets` sans vérification explicite de la taille du tampon de destination permet d'écrire au-delà de l'espace mémoire alloué (**débordement de tampon**), ce qui peut, dans le pire des cas, permettre à un attaquant de modifier le comportement du programme jusqu'à y exécuter son propre code. L'oubli de vérifier le retour de `malloc` expose le programme à un **déréférencement de pointeur nul** dès que l'allocation échoue (mémoire insuffisante), provoquant un plantage exploitable pour un déni de service. Ne pas remettre un pointeur à `NULL` après un `free` crée un risque de **use-after-free** : le programme continue à utiliser une zone mémoire libérée, potentiellement réattribuée entre-temps à une autre donnée, ce qui peut être exploité pour corrompre l'état interne du programme. Une chaîne de format non constante passée directement à `printf` (par exemple `printf(entree_utilisateur)` au lieu de `printf("%s", entree_utilisateur)`) ouvre la porte à une **vulnérabilité de format string**, qui permet de lire, voire d'écrire, à des emplacements arbitraires de la mémoire. Enfin, un calcul de taille non vérifié avant un `malloc` (par exemple `malloc(n * taille_element)` sans vérifier que la multiplication ne dépasse pas la capacité du type entier utilisé) peut provoquer un **débordement d'entier**, aboutissant à une allocation beaucoup plus petite que prévu et donc, de nouveau, à un débordement de tampon lors de l'écriture ultérieure.

6. Limites

Un audit applicatif ponctuel, aussi rigoureux soit-il, ne couvre que l'état du code à un instant donné. Toute évolution ultérieure — un nouveau développeur qui réintroduit sans le savoir un motif de code déjà identifié comme vulnérable, une dépendance mise à jour qui introduit une nouvelle faille, une fonctionnalité ajoutée en urgence sans revue de sécurité — peut réintroduire des vulnérabilités déjà corrigées lors de l'audit précédent, ou en introduire de nouvelles. C'est précisément pour cette raison qu'il est recommandé d'intégrer des contrôles automatisés (SAST, SCA) en continu dans le pipeline de développement, plutôt que de se reposer uniquement sur un audit annuel ou ponctuel qui, entre deux occurrences, laisse une fenêtre de risque potentiellement longue et invisible.

À retenir

- L'OWASP Top 10 est la référence minimale pour structurer un audit applicatif, en combinant des catégories techniques précises et des préoccupations plus transverses (conception, journalisation).
- SAST, DAST et SCA sont complémentaires, pas substituables l'un à l'autre : chacun couvre un angle différent (code source, comportement en exécution, dépendances tierces) et se positionne différemment dans le cycle de développement.
- La méthodologie d'un pentest applicatif progresse de la cartographie globale vers des tests ciblés (authentification, contrôle d'accès, injection, XSS/CSRF, configuration).
- L'audit de code bas niveau (C/C++) cible spécifiquement les erreurs de gestion mémoire et de validation des entrées, des risques propres aux langages sans gestion automatique de la mémoire.
- Un audit ponctuel ne suffit pas dans la durée : l'intégration de contrôles automatisés en continu (CI/CD) réduit la fenêtre de risque entre deux audits formels.

Chapitre 6 — Rapport d'audit et plan de remédiation

Audit organisation et technique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Objectifs du rapport

Le rapport est le **principal livrable** d'un audit de sécurité : c'est lui, et non le travail technique en coulisses, qui sera effectivement lu, discuté en comité de direction et utilisé pour arbitrer des décisions et des budgets. Cette réalité a une conséquence directe sur la façon dont un auditeur doit envisager son travail : un audit techniquement excellent — vulnérabilités réelles identifiées, preuves solides, analyse fine — mais restitué dans un rapport confus, mal structuré ou incompréhensible pour son public perd une grande partie de sa valeur, car les décisions qui devraient en découler ne seront tout simplement pas prises, faute d'avoir été comprises et priorisées correctement. Rédiger un bon rapport d'audit est donc une compétence à part entière, distincte de la compétence technique nécessaire pour mener l'audit lui-même.

2. Structure type d'un rapport d'audit

Un rapport d'audit bien structuré s'adresse simultanément à plusieurs publics dont les besoins diffèrent fortement — une direction qui doit arbitrer des priorités et des budgets sans entrer dans le détail technique, et des équipes techniques qui doivent comprendre précisément quoi corriger et comment. Cette structure en couches successives, de la plus synthétique à la plus détaillée, permet à chaque lecteur de s'arrêter au niveau de détail qui lui est utile :

1. **Synthèse à destination de la direction (*executive summary*)** : une à deux pages maximum, rédigées sans jargon technique, présentant le niveau de risque global et les priorités d'action. C'est souvent la seule section qu'un dirigeant lira intégralement ; elle doit donc pouvoir se suffire à elle-même pour orienter une décision.
2. **Contexte et périmètre** : rappel du mandat, des dates de l'audit, de la méthode employée et des limites explicites de l'audit (ce qui a été testé, et tout aussi important, ce qui ne l'a pas été) — une précision qui protège autant l'organisation auditée que l'auditeur d'une interprétation excessive des résultats.
3. **Méthodologie** : référentiel utilisé (ISO 27001, PTES, OWASP Top 10 selon le type d'audit), mode d'accès (black/grey/white box), outils employés — cette section permet à un lecteur technique d'évaluer la rigueur et la reproductibilité de la démarche.
4. **Constats détaillés**, un par vulnérabilité ou non-conformité identifiée, chacun structuré de façon homogène pour faciliter la lecture croisée entre constats :
5. **titre et catégorie** (par exemple « Injection SQL sur le formulaire de connexion »), qui permet d'identifier immédiatement de quoi il s'agit sans lire le détail ;
6. **criticité**, exprimée via un score CVSS ou une échelle qualitative (Critique/Élevé/Moyen/Faible), pour permettre une priorisation rapide entre constats ;
7. **description technique et preuve de concept** (capture d'écran, extrait de journal, commande utilisée) — sans preuve concrète, un constat reste une simple affirmation, difficile à faire accepter comme fondée ;
8. **impact métier**, qui traduit la conséquence technique en langage compréhensible pour un non-technicien (« un attaquant pourrait accéder à l'ensemble des données clients » plutôt que « injection SQL de type UNION ») ;
9. **recommandation de remédiation**, si possible priorisée et estimée en effort, pour que la correction proposée soit réellement actionnable et pas seulement théorique.
10. **Synthèse des recommandations** sous forme de tableau priorisé, qui rassemble en un seul endroit l'ensemble des actions à mener, classées par criticité — un outil de pilotage direct pour le plan de remédiation qui suit.
11. **Annexes** : détails techniques complets, sorties brutes d'outils, méthodologie détaillée — tout ce qui alourdirait inutilement le corps du rapport sans être nécessaire à sa compréhension générale, mais reste utile pour une équipe technique qui souhaiterait reproduire ou approfondir un constat.

3. Qualifier la criticité

Une bonne pratique consiste à ne jamais utiliser un score technique brut (comme le CVSS vu au chapitre 3) de façon isolée, mais à le combiner systématiquement avec le contexte métier réel de la cible, dans une **matrice de risque** qui croise deux dimensions distinctes :

- la **vraisemblance** : facilité d'exploitation de la faille, exposition réelle du système concerné (accessible depuis Internet ou uniquement depuis un réseau interne fortement restreint, par exemple) ;
- l'**impact** : conséquences sur la confidentialité, l'intégrité, la disponibilité, mais aussi sur la conformité réglementaire de l'organisation.

Cette contextualisation est essentielle car un score technique identique peut correspondre à des risques réels très différents selon l'environnement dans lequel il se manifeste. Une vulnérabilité notée CVSS 9.8 (critique) sur un serveur de test totalement isolé du réseau de production, sans données sensibles et destiné à être détruit sous peu, ne présente pas le même risque réel qu'une vulnérabilité notée CVSS 6.0 (moyenne) mais découverte sur le système de paiement en production, exposé à Internet et traitant quotidiennement des données financières sensibles. Un rapport d'audit de qualité explicite toujours ce raisonnement de contextualisation, plutôt que de se contenter d'un classement mécanique par score CVSS brut.

4. Plan de remédiation

Le plan de remédiation est ce qui transforme chaque recommandation, jusque-là une simple ligne dans un rapport, en une **action suivie et responsabilisée**. Sans ce plan, un rapport d'audit risque de rester un document consulté une fois puis oublié. Chaque ligne du plan associe un constat à une action précise, un responsable identifié nommément (et non simplement « l'équipe informatique »), une échéance réaliste et un statut d'avancement tenu à jour :

Constat	Priorité	Action	Responsable	Échéance	Statut
Mot de passe admin par défaut	Critique	Changer et appliquer une politique de mots de passe forts	Équipe infra	J+7	À faire
Absence de MFA sur le VPN	Élevé	Déployer une authentification à double facteur	RSSI	J+30	En cours
Composant obsolète (CVE connue)	Élevé	Mettre à jour la bibliothèque concernée	Équipe dev	J+15	À faire

L'échéance associée à chaque action est généralement proportionnelle à sa criticité : un constat critique comme un mot de passe administrateur par défaut appelle une correction quasi immédiate (J+7 dans l'exemple ci-dessus), car son exploitation ne requiert quasiment aucune compétence technique particulière, tandis qu'une action plus structurante comme le déploiement d'une authentification multi-facteurs sur l'ensemble des accès VPN nécessite un délai plus réaliste (J+30), le temps de déployer et de tester la solution sans perturber les utilisateurs légitimes. Un plan de remédiation qui fixe des échéances irréalistes, non tenues dans la pratique, perd rapidement toute crédibilité et tout effet incitatif sur les équipes concernées.

5. Contre-audit et suivi

Un rapport d'audit et son plan de remédiation associé ne suffisent pas, à eux seuls, à garantir que les corrections annoncées ont réellement été appliquées et sont effectivement efficaces. C'est le rôle du **contre-audit** (ou audit de suivi), souvent partiel et ciblé spécifiquement sur les constats précédents plutôt que sur l'ensemble du périmètre initial, de vérifier concrètement cette application effective. Cette pratique est explicitement attendue dans les cycles de certification (les audits de surveillance périodiques exigés pour maintenir une certification ISO 27001 en sont un exemple direct), et constitue une preuve tangible de maturité pour l'organisation auditée : une organisation qui accepte et organise elle-même son contre-audit démontre un engagement réel dans sa démarche d'amélioration continue, au-delà de la simple obtention ponctuelle d'un rapport favorable.

6. Communication et postures

La façon dont les résultats d'un audit sont communiqués influence directement la façon dont ils seront reçus et suivis d'effet, indépendamment même de leur qualité technique intrinsèque :

- **Adapter le niveau de détail au public** (direction vs équipes techniques) : c'est précisément la raison d'être de la séparation entre synthèse décisionnelle et annexes techniques évoquée en section 2 — présenter un tableau de score CVSS détaillé à un comité de direction, ou au contraire une synthèse trop générale à une équipe technique qui doit corriger la faille, sont deux erreurs symétriques qui nuisent à l'efficacité du rapport.
- **Rester factuel et non accusatoire** : l'audit évalue un système et ses processus, pas les personnes qui les ont mis en place ou qui les opèrent au quotidien. Un rapport qui personnalise ses constats (« l'administrateur n'a pas fait son travail correctement ») risque de braquer les équipes concernées et de nuire à la coopération nécessaire pour la remédiation, alors qu'un ton factuel centré sur le système favorise une posture constructive.
- **Respecter la confidentialité du rapport** : celui-ci contient potentiellement des informations directement exploitables par un attaquant (détails de vulnérabilités non encore corrigées, cartographie précise de l'infrastructure). Sa diffusion doit être strictement limitée aux parties autorisées, et les données brutes collectées pendant l'audit doivent être détruites après la période contractuelle convenue, conformément aux modalités précisées dans le mandat initial (chapitre 1).

À retenir

- Un rapport d'audit efficace sépare clairement synthèse décisionnelle (executive summary) et détails techniques (constats, annexes), pour s'adresser correctement à des publics aux besoins différents.
- Chaque constat doit être actionnable : criticité contextualisée, preuve concrète, impact métier explicite, recommandation priorisée.
- La criticité technique brute (CVSS) doit toujours être recontextualisée avec l'exposition et l'impact métier réels du système concerné, via une matrice de risque vraisemblance/impact.
- Le plan de remédiation transforme les recommandations en actions suivies, avec responsable nommé, échéance réaliste et statut tenu à jour.
- Le plan de remédiation et le contre-audit transforment l'audit en démarche d'amélioration continue plutôt qu'en une simple photographie isolée, sans effet durable.